

# HNSW, Worked Out by Hand

Why approximate nearest-neighbour search runs in roughly  $\log N$  time instead of  $N$  — the layered “skip-list of graphs” that powers every production vector database — explained with the probabilistic layer assignment and a step-by-step greedy search.

**What this document does.** It builds the intuition for Hierarchical Navigable Small World graphs from two ideas you already know — a skip list and a greedy graph walk — then makes the maths concrete: the exponential layer-assignment probability, the expected number of layers, and why search visits only  $O(\log N)$  nodes. It traces one query through a small multi-layer graph by hand. Numbers from Appendix A.

**This is an intuition PDF:** the goal is a correct mental model and the key scaling law, not a proof of the recall bound (which is empirical).

## 1 The problem: brute force is $O(N)$

To find the nearest vector to a query among  $N$  stored vectors, the exact method compares the query to all  $N$  — linear in the corpus. At  $N = 10^8$  that is 100 million distance computations *per query*. We need sub-linear search, and we are willing to occasionally miss the true nearest neighbour to get it. That trade — *recall for speed* — is what “approximate” (ANN) means.

### Intuition

HNSW is two familiar tricks stacked. **Trick 1 — a navigable graph:** connect each vector to its nearby vectors; to find a neighbour, start anywhere and keep hopping to whichever connected node is closer to the query (greedy descent). **Trick 2 — a skip list of such graphs:** build a few sparse “express layers” on top so you can cover huge distances in a few hops before dropping into the dense bottom layer for the fine search. Coarse-to-fine: cross the continent by plane, the city by taxi, the last block on foot.

## 2 Layer assignment: an exponential decay

Each inserted vector is given a maximum layer  $\ell$  drawn from a geometric distribution. With level multiplier  $m_L$ ,

$$\ell = \lfloor -\ln(U) \cdot m_L \rfloor, \quad U \sim \text{Uniform}(0, 1) \quad P(\ell \geq L) = e^{-L/m_L}.$$

The probability of reaching a layer falls off geometrically: most nodes live *only* on layer 0 (the dense base graph), and each higher layer holds exponentially fewer nodes. With the common choice  $m_L = 1/\ln M$  (where  $M$  is the connections-per-node), the expected number of layers is  $\Theta(\log_M N)$ .

**Concrete counts.** Take  $m_L = 1/\ln 2 \approx 1.443$  so each layer up keeps about half the nodes. Starting from  $N = 16$  base nodes:

| layer | $\approx$ fraction $e^{-L/m_L}$ | $\approx$ nodes (of 16) |
|-------|---------------------------------|-------------------------|
| 0     | 1.00                            | 16                      |
| 1     | 0.50                            | 8                       |
| 2     | 0.25                            | 4                       |
| 3     | 0.125                           | 2                       |
| 4     | 0.0625                          | 1 (entry point)         |

The pyramid has  $\approx \log_2 N = 4$  levels — exactly the skip-list shape.

### 3 Greedy search, traced by hand

A query enters at the *top* layer’s single entry point and, on each layer, greedily hops to the connected node closest to the query until no neighbour is closer — then drops one layer and repeats, using that local optimum as the new entry point. The bottom layer (dense) gives the final answer.

| layer | visited (closest-first hop)     | action                             |
|-------|---------------------------------|------------------------------------|
| 4     | entry $e$                       | only node; descend                 |
| 3     | $e \rightarrow n_7$             | $n_7$ closer; descend              |
| 2     | $n_7 \rightarrow n_3$           | $n_3$ closer; descend              |
| 1     | $n_3 \rightarrow n_5$           | local best; descend                |
| 0     | $n_5 \rightarrow n_4$ (true NN) | no closer neighbour; <b>return</b> |

Five layers, a handful of hops — versus 16 brute-force comparisons here, or  $10^8$  at scale. The express layers let a single hop skip across half the dataset.

#### Mini-example · why it is $O(\log N)$ , not $O(N)$

Each layer roughly *halves* the candidate field (the  $e^{-L/m_L}$  decay), and greedy descent spends only a constant number of hops per layer. With  $\approx \log_M N$  layers and  $O(1)$  work each, total work is  $O(\log N)$ . Going from  $N = 10^6$  to  $N = 10^9$  — a thousand-fold more data — adds only about  $\log_M(1000)$  extra hops, not a thousand-fold more work. That is the entire reason vector search scales.

### 4 The two knobs (and the recall trade)

- $M$  — neighbours stored per node. Larger  $M \Rightarrow$  richer graph, higher recall, more memory ( $O(M)$  per vector) and slower inserts. Typical  $M = 16$ –64.
- **efSearch** — size of the candidate queue kept during search. Larger  $\Rightarrow$  explores more, higher recall, slower query. This is the *runtime* recall/latency dial; raise it until recall is acceptable, then stop.
- **efConstruction** — the same idea at build time; higher means a better graph at the cost of slower indexing.

HNSW does not return guaranteed-correct neighbours; it returns very-probably-correct ones, and these knobs buy recall with latency and memory. Measuring that recall honestly is PDF #12 (Recall@k).

**Intuition**

Failure mode to know: greedy descent can get trapped in a *local* minimum — a node with no closer neighbour that is nonetheless not the global nearest. Larger **efSearch** (keep more candidates, not just the single best) and higher  $M$  (more escape routes) both reduce this. It is why recall climbs smoothly with effort rather than being all-or-nothing.

**Equation cheat-sheet**

$$P(\ell \geq L) = e^{-L/m_L}, \quad m_L = \frac{1}{\ln M}, \quad \text{\#layers} = \Theta(\log_M N)$$

$$\text{search} = O(\log N) \text{ hops}, \quad \text{recall} \uparrow \text{ with } M \uparrow, \text{ efSearch} \uparrow \text{ (latency } \uparrow)$$

**Appendix A · Layer-distribution sketch**

```
import numpy as np

mL = 1/np.log(2)                # level multiplier (~1.443)
def sample_level(u):             # u ~ Uniform(0,1)
    return int(-np.log(u) * mL)

# Expected fraction of nodes reaching layer L: exp(-L/mL)
for L in range(5):
    print(L, round(np.exp(-L/mL), 4))  # 1.0, 0.5, 0.25, 0.125, 0.0625
```

Internal learning material. Layer fractions reproducible from the appendix. v1.0